In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries Uploaded Successfully!")
```

```
Libraries Uploaded Successfully!
```

In [2]:
```
df=pd.read_csv("Student Social Media And Mental Health Impact.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	Age	Gender	Country	Academic_Level	Most_Used_Platform	Purpose_Of_Use	Avg_Daily
0	21	Male	Other	Undergraduate	Facebook	Networking	
1	23	Female	Other	Graduate	LinkedIn	Education	
2	22	Male	Canada	Undergraduate	Instagram	Entertainment	
3	18	Male	Other	High School	Snapchat	Entertainment	
4	24	Female	Other	Graduate	Facebook	Networking	

In [4]:
```
df.shape
```

Out[4]:
```
(5000, 13)
```

In [5]:
```
df.columns
```

Out[5]:
```
Index(['Age', 'Gender', 'Country', 'Academic_Level', 'Most_Used_Platform',
       'Purpose_Of_Use', 'Avg_Daily_Usage_Hours', 'Daily_Unlocks',
       'Study_Hours', 'Physical_Activity_Hours', 'Sleep_Hours_Per_Night',
       'Stress_Level', 'Mental_Health_Score'],
      dtype='object')
```

In [6]:
```
df.describe()
```

Out[6]:

	Age	Avg_Daily_Usage_Hours	Daily_Unlocks	Study_Hours	Physical_Activity_H
count	5000.00000	5000.000000	5000.000000	5000.000000	5000.000
mean	20.82180	5.078460	171.452600	3.008420	1.751
std	1.73662	1.653913	42.858254	1.637018	0.668
min	18.00000	1.000000	62.000000	0.300000	-0.400
25%	19.00000	3.800000	140.000000	1.500000	1.300
50%	21.00000	5.000000	171.000000	2.800000	1.700
75%	22.00000	6.300000	204.000000	4.200000	2.200
max	24.00000	8.800000	273.000000	8.300000	4.100

In [7]:
```
df.isnull().sum()
```

Out[7]:
```
Age                       0
Gender                    0
Country                   0
Academic_Level            0
Most_Used_Platform        0
Purpose_Of_Use            0
Avg_Daily_Usage_Hours     0
Daily_Unlocks             0
Study_Hours               0
Physical_Activity_Hours   0
Sleep_Hours_Per_Night     0
Stress_Level              0
Mental_Health_Score       0
dtype: int64
```

In [8]:
```
df.duplicated().sum()
```

Out[8]:
```
np.int64(2)
```

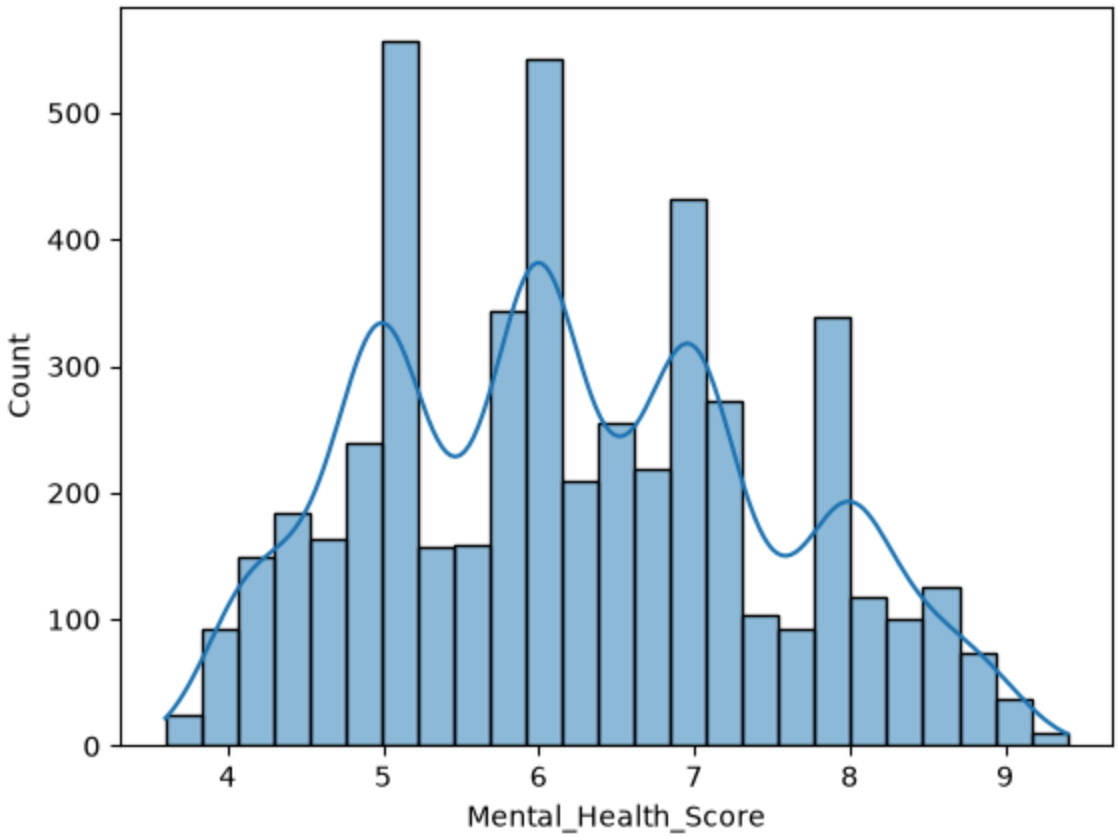
In [9]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 13 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   Age                      5000 non-null   int64  
 1   Gender                   5000 non-null   object 
 2   Country                  5000 non-null   object 
 3   Academic_Level           5000 non-null   object 
 4   Most_Used_Platform       5000 non-null   object 
 5   Purpose_Of_Use           5000 non-null   object 
 6   Avg_Daily_Usage_Hours    5000 non-null   float64
 7   Daily_Unlocks            5000 non-null   int64  
 8   Study_Hours              5000 non-null   float64
 9   Physical_Activity_Hours  5000 non-null   float64
 10  Sleep_Hours_Per_Night    5000 non-null   float64
 11  Stress_Level             5000 non-null   object 
 12  Mental_Health_Score      5000 non-null   float64
dtypes: float64(5), int64(2), object(6)
memory usage: 507.9+ KB
```

Exploratory Data Analysis(EDA)

In [10]:
```
#1. Distribution of the Targets
sns.histplot(df["Mental_Health_Score"], kde=True)
```

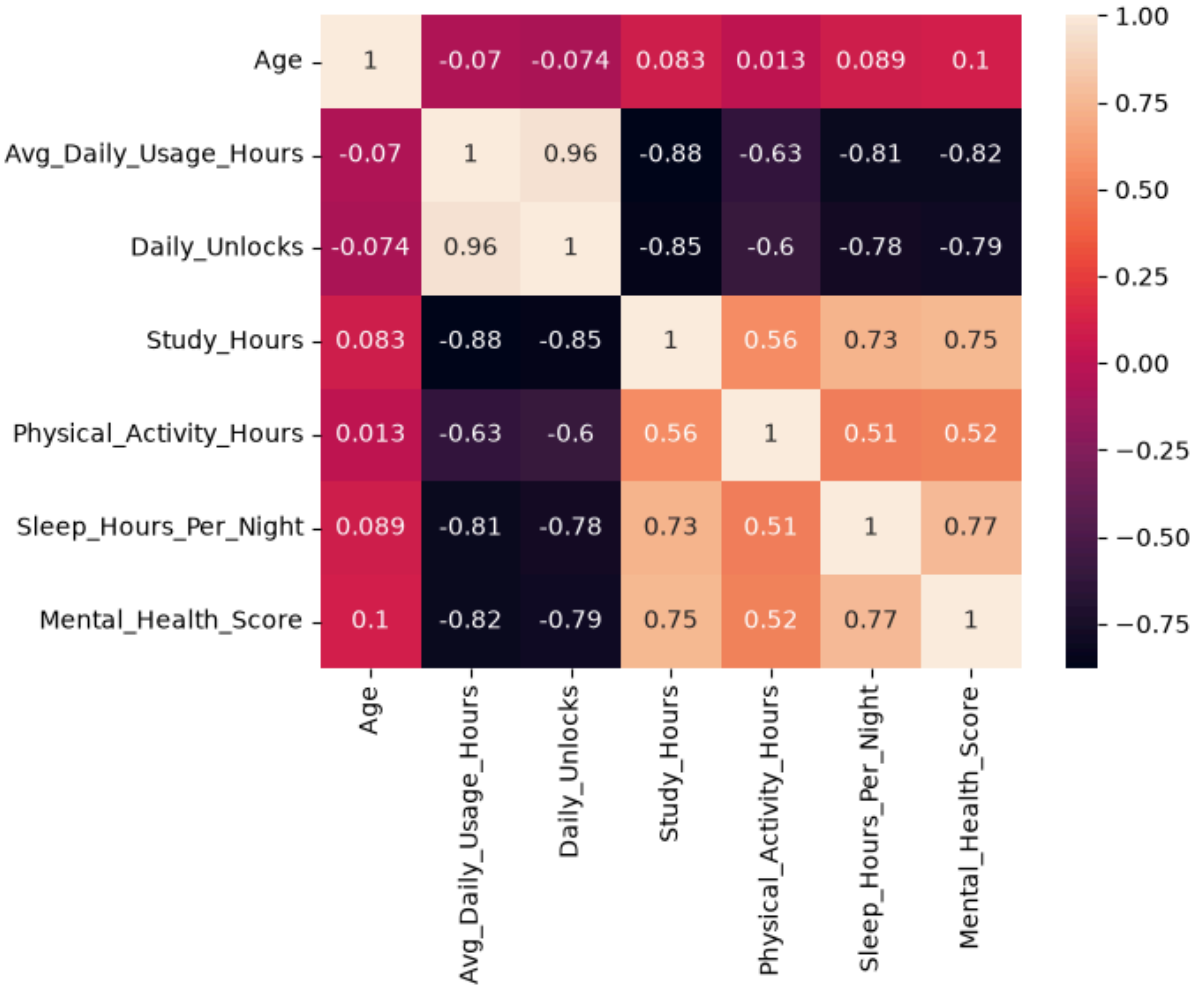
Out[10]: <Axes: xlabel='Mental_Health_Score', ylabel='Count'>



In [11]:
```
# 2.Corelation Heatmap

sns.heatmap(df.corr(numeric_only=True), annot=True)
```

Out[11]: <Axes: >



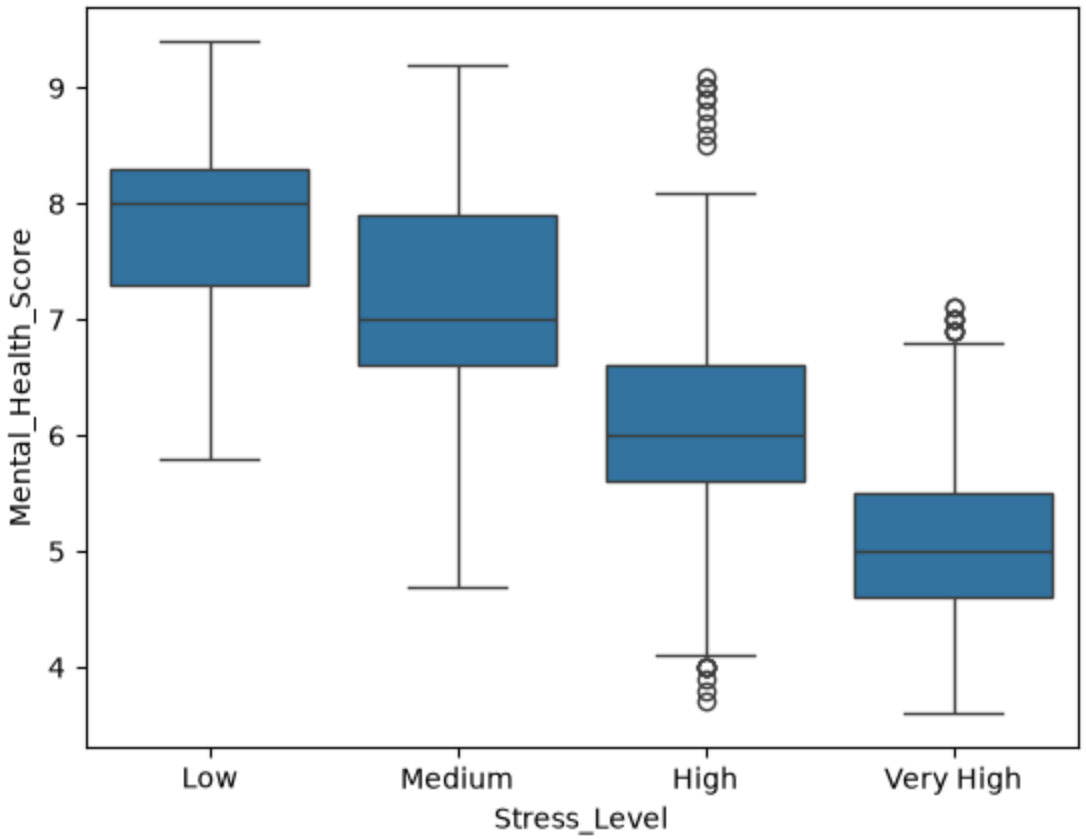
In [12]:
```
#3. Stress Level vs Mental Health Score
```

In [13]:
```
df['Stress_Level'].unique()
```

Out[13]: array(['Medium', 'Low', 'Very High', 'High'], dtype=object)

In [14]:
```
order = ['Low', 'Medium', 'High', 'Very High']
sns.boxplot(x='Stress_Level', y='Mental_Health_Score', data=df, order=order)
```

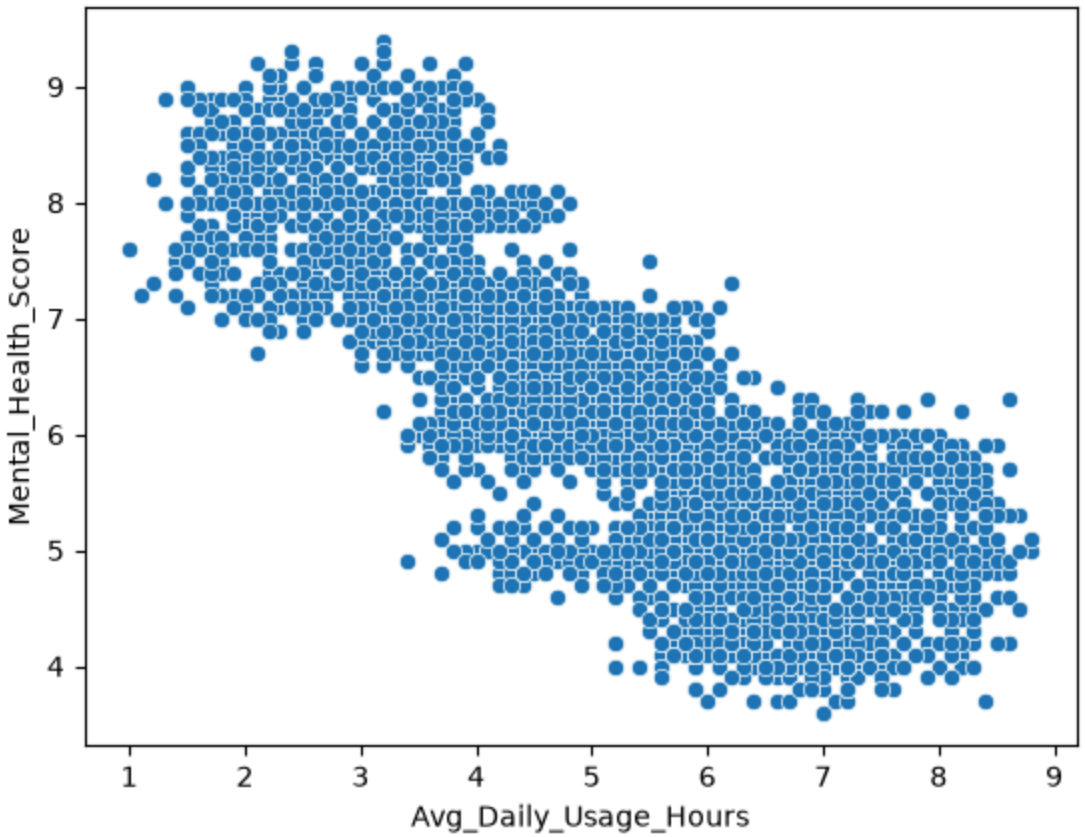
Out[14]: <Axes: xlabel='Stress_Level', ylabel='Mental_Health_Score'>



In [15]:
```
#4. Dailly usage Hours vs Mental Health
```

In [16]:
```
sns.scatterplot(x='Avg_Daily_Usage_Hours', y= 'Mental_Health_Score', data=df)
```

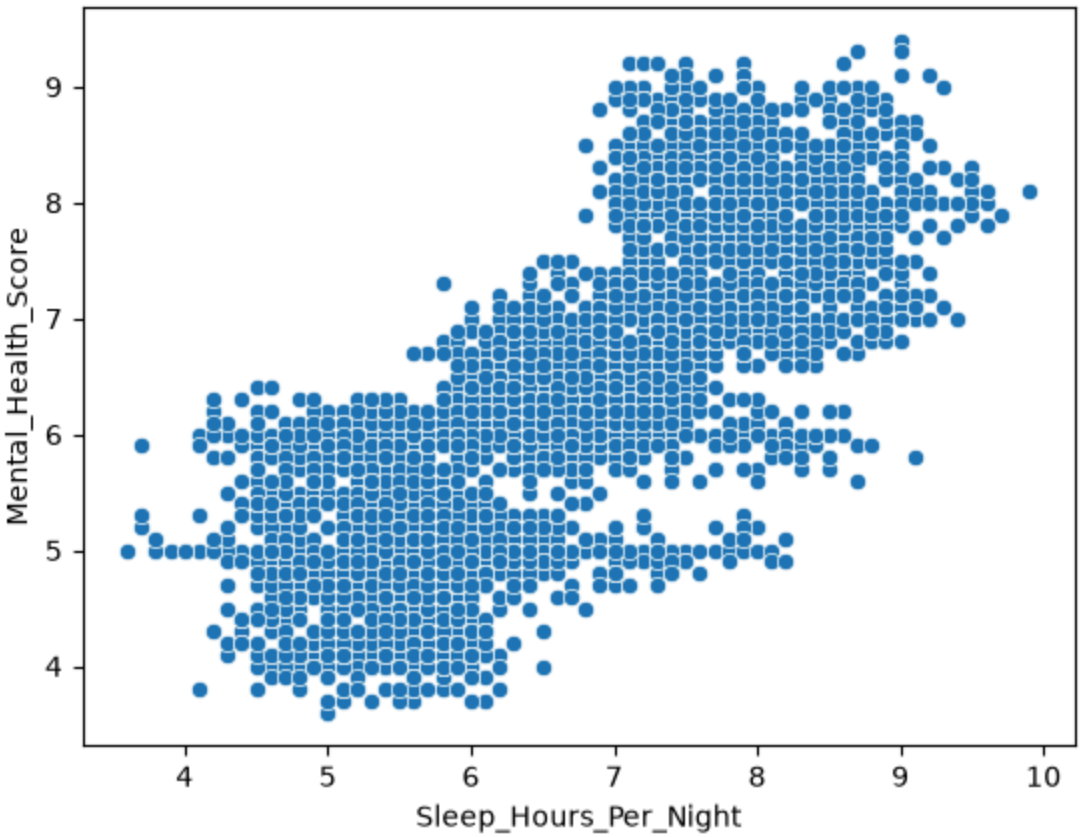
Out[16]:
```
<Axes: xlabel='Avg_Daily_Usage_Hours', ylabel='Mental_Health_Score'>
```



In [17]:
```
# 5. Sleep Hour vs Mental health Score
```

In [18]:
```
sns.scatterplot(x='Sleep_Hours_Per_Night', y='Mental_Health_Score', data=df)
```

Out[18]:
```
<Axes: xlabel='Sleep_Hours_Per_Night', ylabel='Mental_Health_Score'>
```



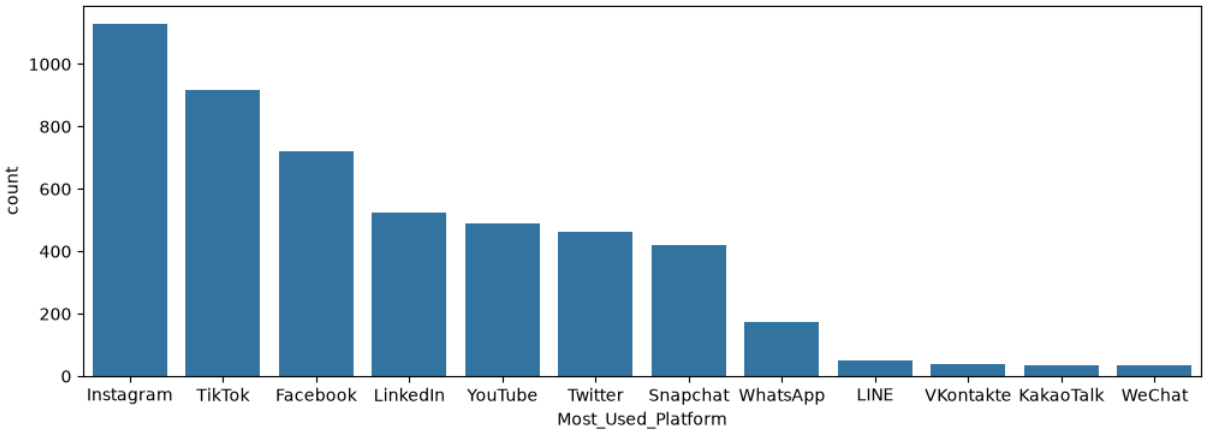
In [19]:
```
#6. Most Used Platform(count)
```

In [20]:
```
df['Most_Used_Platform'].value_counts()
```

Out[20]:
```
Most_Used_Platform
Instagram    1130
TikTok        918
Facebook      719
LinkedIn      523
YouTube       491
Twitter       462
Snapchat      420
WhatsApp      175
LINE           50
VKontakte      40
KakaoTalk      36
WeChat         36
Name: count, dtype: int64
```

In [21]:
```
plt.figure(figsize=(12,4))
sns.countplot(x='Most_Used_Platform', data=df, order=df['Most_Used_Platform'].value
```

Out[21]:
```
<Axes: xlabel='Most_Used_Platform', ylabel='count'>
```



Checking Outliers

In [22]:
```
num_features = df.select_dtypes(include='number')  #int64 or float64
Q1 = num_features.quantile(0.25)
Q3 = num_features.quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5*IQR
upper_bound = Q3 + 1.5*IQR

outliers = (num_features < lower_bound) | (num_features > upper_bound)
outliers.sum()
```

Out[22]:
```
Age                        0
Avg_Daily_Usage_Hours      0
Daily_Unlocks              0
Study_Hours                2
Physical_Activity_Hours    22
Sleep_Hours_Per_Night      0
Mental_Health_Score        0
dtype: int64
```

Data Cleaning

In [23]:
```
#1. To drop duplicates
df.duplicated().sum()
```

Out[23]:
```
np.int64(2)
```

In [24]:
```
df = df.drop_duplicates()
```

In [25]:
```
df.duplicated().sum()
```

Out[25]:
```
np.int64(0)
```

In [26]:
```
df.describe()
```

Out[26]:

	Age	Avg_Daily_Usage_Hours	Daily_Unlocks	Study_Hours	Physical_Activity_H
count	4998.000000	4998.000000	4998.000000	4998.000000	4998.00
mean	20.822129	5.078491	171.455582	3.008403	1.75
std	1.736774	1.654097	42.859829	1.636831	0.66
min	18.000000	1.000000	62.000000	0.300000	-0.40
25%	19.000000	3.800000	140.000000	1.500000	1.30
50%	21.000000	5.000000	171.000000	2.800000	1.70
75%	22.000000	6.300000	204.000000	4.200000	2.20
max	24.000000	8.800000	273.000000	8.300000	4.10

In [27]:
```
#2. Converting negative or unrealistic value to zero
df['Physical_Activity_Hours'] = df['Physical_Activity_Hours'].clip(lower=0)
```

In [28]:
```
df.describe()
```

Out[28]:

	Age	Avg_Daily_Usage_Hours	Daily_Unlocks	Study_Hours	Physical_Activity_H
count	4998.000000	4998.000000	4998.000000	4998.000000	4998.00
mean	20.822129	5.078491	171.455582	3.008403	1.75
std	1.736774	1.654097	42.859829	1.636831	0.66
min	18.000000	1.000000	62.000000	0.300000	0.00
25%	19.000000	3.800000	140.000000	1.500000	1.30
50%	21.000000	5.000000	171.000000	2.800000	1.70
75%	22.000000	6.300000	204.000000	4.200000	2.20
max	24.000000	8.800000	273.000000	8.300000	4.10

Skewness

In [29]:
```
num_cols = df.select_dtypes(include='number')
num_cols.skew()
# near to zero ---> Centralized(0.01.0.002)
# negative(less than zer0)---> Left skewed(-1.56)
# positive( greater than 0) ---> Right Skewed(1.256)
```

Out[29]:
```
Age                         0.155008
Avg_Daily_Usage_Hours       0.005575
Daily_Unlocks               0.002309
Study_Hours                 0.436125
Physical_Activity_Hours     0.053288
Sleep_Hours_Per_Night       0.123919
Mental_Health_Score         0.207086
dtype: float64
```

Feature Engineering

In [30]:
```
df['Stress_Level'].unique()
```

Out[30]:
```
array(['Medium', 'Low', 'Very High', 'High'], dtype=object)
```

In [31]:
```
top_countries = df['Country'].value_counts().index[:10].tolist()
```

In [32]:
```
top_countries
```

Out[32]:
```
['Other',
 'India',
 'USA',
 'Canada',
 'Australia',
 'UK',
 'Germany',
 'Turkey',
 'Mexico',
 'France']
```

In [33]:
```
def group_countries(country):
    if country in top_countries:
        return country
    else:
        return 'Other'
```

In [34]:
```
df['grouped_country']=df['Country'].apply(group_countries)
```

In [35]:
```
df['grouped_country'].value_counts()
```

Out[35]:
```
grouped_country
Other        3231
India         389
USA           354
Canada        230
Australia     198
UK            185
Germany       136
Mexico         94
Turkey         94
France         87
Name: count, dtype: int64
```

Encoding Strategy

Before we jump into code, let's decide *how* each categorical column should be encoded — this decision matters more than the code itself.

- `Stress_Level` → **Ordinal Encoding.** Its categories have a real, meaningful order: Low < Medium < High < Very High. We already saw in EDA (section 4.3) that the score drops step by step as stress increases — encoding it as 0, 1, 2, 3 preserves that order for the model.
- `Gender`, `Academic_Level`, `Most_Used_Platform`, `Purpose_Of_Use`, `Country_Grouped` → **One-Hot Encoding.** These categories have **no natural order** — "Instagram" isn't "greater than" "LinkedIn". One-hot encoding creates a separate 0/1 column per category so the model doesn't accidentally assume a false ranking.

We'll implement both of these inside a single `ColumnTransformer` in the next step, rather than doing it manually column by column — that keeps preprocessing consistent and reusable.

9. Train-Test Split

Why we split: we need data the model has never seen to honestly check how well it generalizes. If we evaluate on the same data we trained on, the model can just "memorize" the answers and look artificially good.

Common ratios: 80/20 or 70/30 are both standard. We'll use an 80/20 split here — with 5,000 rows, that still leaves 1,000 rows for a solid, reliable test.

We split **before** fitting any preprocessing — this is the single most common beginner mistake (called data leakage), and we'll avoid it by fitting everything only on the training set.

In [36]:
```
from sklearn.model_selection import train_test_split


skewed_col = ['Study_Hours']
other_numeric_cols = ['Age', 'Avg_Daily_Usage_Hours', 'Daily_Unlocks', 'Physical_Ac

ordinal_col=['Stress_Level']
normal_col=['Gender', 'Academic_Level', 'Most_Used_Platform', 'Purpose_Of_Use', 'gr

feature_col = skewed_col + other_numeric_cols + ordinal_col + normal_col
x = df[feature_col]
y = df['Mental_Health_Score']

x_train, x_test, y_train, y_test= train_test_split(x, y, test_size=0.30, random_sta
```

10. Preprocessing using ColumnTransformer

Our columns need **different treatment**:

- `Study_Hours` (the skewed one) → impute → `log1p` transform → scale
- The other numeric columns → impute → scale (no skew to fix)
- `Stress_Level` → impute → `OrdinalEncoder` with an **explicit order**
- The nominal columns → impute → `OneHotEncoder`

We include a `SimpleImputer` in every branch even though this dataset has zero missing values right now — it's a safety net. Real-world data (and our future API's incoming requests) won't always be this clean, and a pipeline that assumes "no missing values ever" is a pipeline that breaks in production.

`ColumnTransformer` glues all of this into **one object** that applies the right transformation to the right column type in a single `.fit()` / `.transform()` call — no manual column-by-column juggling.

In [37]:
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler, OrdinalEncod
from sklearn.compose import ColumnTransformer

#1. Skewed Features
skewed_pipeline = Pipeline(steps=[
    ('log_transform', FunctionTransformer(np.log1p)),
    ('scale', StandardScaler())
])

#2. Numeric Features
plain_numeric_pipeline = Pipeline([
    ('scale', StandardScaler())
])

#3. Ordinal
ordinal_pipeline = Pipeline(steps=[
    ('encode', OrdinalEncoder(categories=[['Low', 'Medium', 'High', 'Very High']]))
])

#4.Nominal Features
nominal_pipeline= Pipeline(steps=[
    ('encode', OneHotEncoder(handle_unknown='ignore'))
])


preprocessor = ColumnTransformer(transformers=[
    ('Skewed_Pipeline', skewed_pipeline, skewed_col),
    ('plain_numeric_pipeline', plain_numeric_pipeline, other_numeric_cols),
    ('Ordinal', ordinal_pipeline, ordinal_col),
    ('Normal', nominal_pipeline, normal_col)
])
```

11. Build a Pipeline

Why Pipeline matters: a `Pipeline` chains preprocessing and the model into a single object. Calling `.fit()` once does both steps in the correct order, and calling `.predict()` on brand-new raw data automatically applies the exact same preprocessing that was used during training — no risk of forgetting a step or applying it inconsistently.

Why companies prefer this: when this model gets deployed (which we're doing in Part 2 with FastAPI), the API doesn't need to know *anything* about scaling, encoding, or log transforms — it just loads one saved pipeline object and calls `.predict()` on raw input. That's a huge reduction in what can go wrong in production.

We'll build two pipelines below — one per model — so we can fairly compare them.

In [ ]:

12. Model Building

We're comparing **two models** here because the comparison itself teaches something useful — a plain Linear Regression baseline versus a Random Forest, which can capture the non-linear patterns we spotted back in EDA (section 4.4).

12.1 — Baseline: Linear Regression

In [38]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error


lr_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

lr_pipeline.fit(x_train, y_train)
lr_preds_testing = lr_pipeline.predict(x_test)
lr_preds_training = lr_pipeline.predict(x_train)


lr_r2_testing = r2_score(y_test, lr_preds_testing)
lr_r2_training = r2_score(y_train, lr_preds_training)
lr_mae = mean_absolute_error(y_test, lr_preds_testing)


print(f"Accuracy of Training: {lr_r2_training}")
print(f"Accuracy of Testing: {lr_r2_testing}")
print(f"MAE {lr_mae}")
```

```
Accuracy of Training: 0.7236771199387539
Accuracy of Testing: 0.7397944617433021
MAE 0.5361775634720183
```

12.2 — Random Forest (default settings)

Same preprocessor, different model at the end — that's the entire point of building the pipeline once.

In [39]:
```
from sklearn.ensemble import RandomForestRegressor

rf_pipeline= Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('random forest', RandomForestRegressor(random_state=42) )
])

rf_pipeline.fit(x_train, y_train)

rf_preds = rf_pipeline.predict(x_test)
rf_preds_train = rf_pipeline.predict(x_train)

rf_r2_testing = r2_score(y_test, rf_preds)
rf_r2_training = r2_score(y_train, rf_preds_train)
rf_mae = mean_absolute_error(y_test, rf_preds)

print(f"Accuracy of Training: {rf_r2_training}")
print(f"Accuracy of Testing: {rf_r2_testing}")
print(f"MAE: {rf_mae}")
```

```
Accuracy of Training: 0.9809069341581819
Accuracy of Testing: 0.8780167461546369
MAE: 0.34650236666666673
```

12.3 — Hyperparameter Tuning the Random Forest

When tuning a `Pipeline`, each parameter needs to be prefixed with the step name (`regressor__`) so `RandomizedSearchCV` knows which part of the pipeline it belongs to. We use `RandomizedSearchCV` instead of a full grid search — it samples a fixed number of combinations instead of testing every single one, which is far faster while still finding a strong set of parameters.

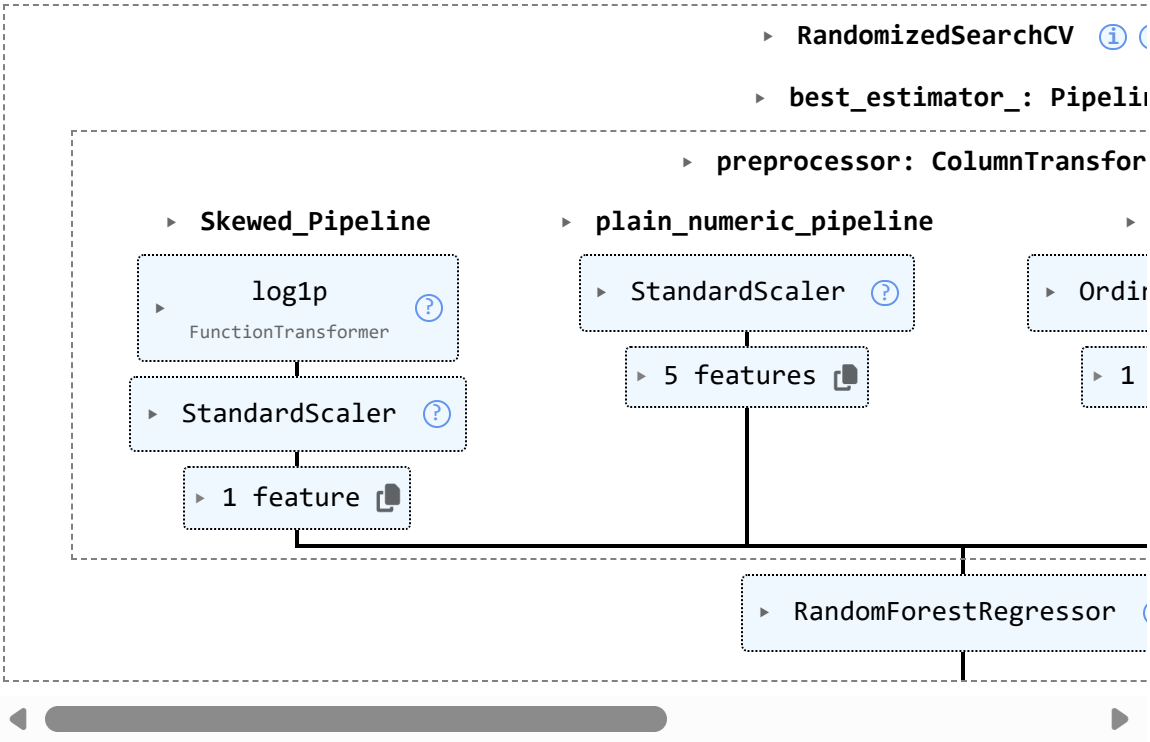
In [40]:
```
from sklearn.model_selection import RandomizedSearchCV
param_grid = {
    'random forest__n_estimators'       : [100,200,300],
    'random forest__max_depth'         : [5,10,15],
    'random forest__min_samples_split' : [2,5,10],
    'random forest__min_samples_leaf'  : [1,2,4]
}

random_search = RandomizedSearchCV(
    estimator = rf_pipeline,
    param_distributions = param_grid,
    n_iter=15,
    cv=5,
    scoring='r2',
    random_state=42,
```

```
    n_jobs=-1 #processor free
)


random_search.fit(x_train, y_train)
```

Out[40]:



In [41]:
```
random_search.best_params_
```

Out[41]:
```
{'random forest__n_estimators': 200,
 'random forest__min_samples_split': 5,
 'random forest__min_samples_leaf': 2,
 'random forest__max_depth': 15}
```

In [42]:
```
rf_best_pipeline = random_search.best_estimator_
#rf_best_pipeline
rf_best_preds = rf_best_pipeline.predict(x_test)
print(f"R2 Score for after hyperparameter tunning {r2_score(y_test, rf_best_preds)}
print(f"MAE Score for random forest after hyperparameter tunning {mean_absolute_err
```

```
R2 Score for after hyperparameter tunning 0.8652257547733472
MAE Score for random forest after hyperparameter tunning 0.36866505007865863
```

13. Model Evaluation

Metrics, explained simply:

- **R^2 Score** — how much of the variation in mental health scores our model explains, from 0 (useless) to 1 (perfect). 0.85 means the model explains 85% of the pattern.
- **MAE (Mean Absolute Error)** — on average, how far off our prediction is, in the original units (score points). Easy to explain to a non-technical person.

- **RMSE (Root Mean Squared Error)** — similar to MAE, but penalizes big mistakes more heavily. Useful for spotting whether a model makes a few very wrong predictions.

Cross-validation R^2 (from tuning, above) tells us how good the model is across many different train/validation splits — the real, honest test is still the untouched test set below.

In [43]:
```
from sklearn.metrics import mean_squared_error

# Calculate RMSE for Linear Regression
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_preds_testing))

# Calculate RMSE for default Random Forest
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_preds))

# Calculate RMSE for tuned Random Forest
rf_tuned_preds          = random_search.best_estimator_.predict(x_test)
rf_tuned_rmse           = np.sqrt(mean_squared_error(y_test, rf_tuned_preds))
rf_tuned_training_preds = random_search.best_estimator_.predict(x_train)
r2_tuned_training       = r2_score(y_train, rf_tuned_training_preds)
rf_tuned_mae            = mean_absolute_error(y_test, rf_tuned_preds)
rf_tuned_r2             = r2_score(y_test, rf_tuned_preds)


# Create a DataFrame to consolidate results
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Random Forest (default)', 'Random Forest (tuned
    'R2': [lr_r2_testing, rf_r2_testing, rf_tuned_r2],
    'Training R2': [lr_r2_training, rf_r2_training, r2_tuned_training],
    'MAE': [lr_mae, rf_mae, rf_tuned_mae],
    'RMSE': [lr_rmse, rf_rmse, rf_tuned_rmse]
})

print(results)
```

```
                   Model        R2  Training R2       MAE      RMSE
0      Linear Regression  0.739794     0.723677  0.536178  0.676032
1  Random Forest (default)  0.878017     0.980907  0.346502  0.462870
2    Random Forest (tuned)  0.865226     0.954656  0.368665  0.486533
```

14. Save the Model

We save the **entire pipeline** — preprocessing and model together — as a single file, not just the raw model. **Why:** if we only saved the Random Forest object, our FastAPI backend would need to manually recreate every encoding and scaling step by hand at prediction time, which is fragile and easy to get wrong. Saving the full pipeline means the backend just loads one file and calls `.predict()` on raw input — the pipeline handles everything internally, exactly the same way it did during training.

In [44]:
```
import joblib
joblib.dump(rf_pipeline, 'Mental_Health_Model.pkl')
```

Out[44]:
```
['Mental_Health_Model.pkl']
```

Next steps (Part 2):

build a FastAPI backend with Pydantic request validation that loads `mental_health_pipeline.pkl` and exposes a `/predict` endpoint, then connect a simple frontend, then deploy both so this becomes a live, clickable demo.

In [45]:
```
df.columns.tolist()
```

Out[45]:
```
['Age',
 'Gender',
 'Country',
 'Academic_Level',
 'Most_Used_Platform',
 'Purpose_Of_Use',
 'Avg_Daily_Usage_Hours',
 'Daily_Unlocks',
 'Study_Hours',
 'Physical_Activity_Hours',
 'Sleep_Hours_Per_Night',
 'Stress_Level',
 'Mental_Health_Score',
 'grouped_country']
```

In [ ]:
```

```